



Software Engineering Project

Project Documentation

Team Members:

Farah Khaled Nabil (2023/05623)
Farida Ahmed Seif (2023/05682)
Farida Hossam Hamdy (2023/04152)
Joanna Hazem Ibrahim (2023/01073)
Youliana Remone Nassef (2023/08045)

Contents

1	Preface	3
2	Introduction	3
3	Similar Systems	4
3.1	LinkedIn	4
3.2	Facebook	4
3.3	Uber	4
4	Glossary	4
5	Functional User Requirements	6
5.1	User	6
5.2	Admin	6
6	Functional System Requirements	7
7	Non-functional Requirements	10
7.0.1	Dependability requirements	12
7.0.2	Security requirements	12
7.0.3	Usability requirements	13
7.1	Organizational requirements	13
7.1.1	Development requirements	13
7.1.2	Operational requirements	14
7.1.3	Environmental requirements	14
7.2	External requirements	14
7.2.1	Interoperability requirements	14
7.2.2	Legislative requirements	15
7.2.3	Ethical requirements	15
7.2.4	Regulatory requirements	15
8	Scenarios	15
9	Use Cases	23
10	Form-Based Specification	25
10.1	Sign Up / Sign In	25
10.2	Post a Job	26
10.3	View My Posted Jobs	26

10.4 Accept / Decline Job Requests	27
10.5 Search / Browse Jobs	27
10.6 Direct Messages	28
10.7 Posting a Community Post	28
10.8 Report an Issue	29
10.9 Remove Inappropriate Content	29
11 Tabular Specification	30

Preface

This project presents the development of Shaghalny, a social media-based web platform designed to help students and teenagers find quick, local job opportunities and short-term tasks. The idea was inspired by the growing need for young people to gain financial independence, work experience, and community involvement in an easy and accessible way.

Shaghalny connects users with nearby job opportunities such as pet walking, tutoring, delivery, or simple one-time tasks through an interactive map and posting system. It combines the functionality of a job board with the engagement of a social media platform, allowing users to interact through a community feed, share achievements, and communicate directly.

In addition to supporting students, the platform benefits local residents and small businesses who can easily post temporary jobs and select suitable applicants. The system also includes profile ratings, privacy controls, and an admin dashboard to ensure a safe and efficient user experience.

The project aims to provide a practical, modern, and youth-friendly solution for connecting people and opportunities within their local communities — encouraging responsibility, independence, and collaboration among young users.

Introduction

This software engineering project, Shaghalny, is being developed to address a common challenge within local communities: the difficulty of finding reliable help for small-scale jobs. At the same time, it provides a solution for students who need flexible work opportunities. Shaghalny is a social media website that acts as the bridge between these two needs, creating a hyper-local marketplace for one-time or part-time tasks.

The platform's design is inspired by a blend of successful concepts: it uses a live, map-based discovery system similar to Uber for locating opportunities [?], while incorporating a professional profile and review system akin to LinkedIn [?] to build user reputation. The scope of this project is to build the core functional website. This includes user profiles, a system for posting and applying for jobs, a “community” page for users to share their experiences, a direct messaging feature, and privacy controls.

The website will support two main user types:

- **User:** This includes Requesters who post jobs and review applicants, and Doers (primarily students) who apply for and complete those jobs.
- **Admin:** This is the backend role for the project team, responsible for managing users, moderating content, handling user reports, and generating analytics on

platform engagement.

Similar Systems

LinkedIn

LinkedIn provides a professional feed where users can post updates, share experiences, and interact with others in their field. Our community page is inspired by LinkedIn’s engagement model, allowing users to share achievements or feedback about job experiences, interact through comments, and build connections. Unlike LinkedIn, our platform focuses on short-term, flexible, and local tasks [?].

Facebook

The community feature draws inspiration from Facebook and its Groups, where users can post local job opportunities and engage in discussions. However, unlike Facebook Groups, our system introduces structured job management and applicant-handling tools, while still allowing users to interact casually and build community connections [?].

Uber

Our platform also shares similarities with Uber in user interaction and functionality. The integrated map displays nearby job opportunities as pop-ups, similar to Uber’s live map of available drivers. Both platforms include a rating system for building mutual trust between users [?].

Glossary

Term	Definition
User	Any individual registered on the platform, including job posters, and applicants (doers).
Requester (Job Poster)	A user who posts a job or task that they need help with.
Applicant	A user who applies for and completes posted jobs or tasks.

Admin	The system administrator responsible for monitoring content, managing users, handling reports, and ensuring platform security and functionality.
Job Post	A task or short-term job listing created by a user, including title, description, pay rate, and location.
Application	A submission from an applicant expressing interest in a specific job post.
Community Page	A social feed where users can share posts, updates, or experiences and interact through likes, comments, or shares.
Profile Rating	A score (out of 5) reflecting user reliability and performance, based on feedback from completed tasks.
Verification	The process of confirming user authenticity through email and National ID.
Report	A complaint submitted by a user against another user, post, or issue for admin review.
Direct Message (DM)	A private communication channel between users.
Notification	A system message alerting users about new messages, job updates, or admin responses.
Dashboard	The admin interface used to monitor system data, moderate content, and manage user accounts.
Map View	The interactive map displaying nearby job opportunities.
Guest	A non-registered visitor who can browse limited parts of the site but cannot post or apply for jobs.
Job Status	The current state of a job post (e.g., Open, In Progress, Closed).
Job Category	A classification of job types such as tutoring, delivery, pet-sitting, or cleaning.
Support Center	A system feature that allows users to get help, report issues, or contact admins.

Privacy Settings	User-defined controls over which profile information is visible to others.
-------------------------	--

Functional User Requirements

User

- The user can create an account, log in, and reset their password.
- The user can view, edit, and manage their profile information.
- The user can post new jobs, apply for jobs, and manage job listings.
- The user can view job recommendations and search for nearby jobs.
- The user can view their work history, jobs they have posted, jobs they have applied for, and job requests they have received from other users.
- The user can interact with other users by:
 1. Following/unfollowing them.
 2. Liking their posts or job updates.
 3. Sending direct messages.
 4. Rating or reviewing their performance after collaboration.
- The user can enter the job title, description, payment amount, location, and time limit for completing the job.
- The user can report an issue related to jobs, posts, or other users.

Admin

- The admin can manage user accounts by reviewing, suspending, or deleting users.
- The admin can moderate content by removing inappropriate or reported posts.
- The admin can control platform features by enabling or disabling specific functionalities.
- The admin can analyze trends by generating reports on user engagement, popular content, and overall platform activity.
- The admin can review and respond to user reports and support requests.

Functional System Requirements

1. Job Post Function

- 2.1** The system shall allow registered users to create job posts containing details such as job title, description, category, pay rate, job type (one-time, part-time, full-time), and location.
- 2.2** The system shall validate all fields in the job posting form before submission (e.g., pay rate must be a positive numeric value, title and description cannot be empty).
- 2.3** The system shall allow users to upload an optional photo or document related to the job.
- 2.4** The system shall display a confirmation message upon successful posting.
- 2.5** The system shall allow users to edit or delete their job posts at any time before the job is marked as closed.
- 2.6** The system shall allow users to browse available job listings and filter them by category, pay rate, job type, or location.
- 2.7** The system shall support keyword-based search for jobs.
- 2.8** The system shall allow users to apply for any job that is open.
- 2.9** The system shall notify job posters whenever new applications are submitted.
- 2.10** The system shall allow job posters to view, accept, or reject applications.
- 2.11** Once a job is filled, the system shall update its status to “In Progress” or “Closed.”
- 2.12** The system shall store all job-related actions (post creation, updates, acceptance, and rejection) in the database and system logs.

2. Community Function

- 3.1** The system shall allow users to create community posts consisting of text and optional media (image, video, or attachment).
- 3.2** The system shall display community posts chronologically or by popularity in the community feed.

- 3.3 The system shall allow other users to like, comment on, or share community posts.
- 3.4 The system shall store the total number of likes, comments, and shares for each post.
- 3.5 The system shall allow users to report community posts or comments that are inappropriate, spam, or offensive.
- 3.6 The system shall notify admins whenever a report is made.
- 3.7 The system shall allow admins to review, hide, or delete posts or comments that violate platform policies.
- 3.8 The system shall log all content moderation actions for audit purposes.

3. Reviews and Feedback

- 4.1 The system shall allow users to leave reviews and ratings (out of 5) after the completion of a job.
- 4.2 The system shall validate that only users who participated in a job (poster or worker) can leave reviews for each other.
- 4.3 The system shall display reviews on user profiles, along with average rating scores.
- 4.4 The system shall allow users to report fake, offensive, or irrelevant reviews.
- 4.5 The system shall allow admins to remove or edit reviews that are flagged and verified as inappropriate.

4. Messaging Function

- 5.1 The system shall allow users to send and receive direct messages related to job listings or general inquiries.
- 5.2 The system shall maintain a conversation history for each user pair.
- 5.3 The system shall notify users when they receive a new message (via in-website notification or email).
- 5.4 The system shall allow users to mark messages as read or unread.

- 5.5 The system shall allow users to report or block other users for inappropriate messages or harassment.
- 5.6 The system shall prevent sending or receiving messages from blocked users.

5. Admin Dashboard

- 6.1 The system shall provide a dedicated admin dashboard accessible only to users with admin privileges.
- 6.2 The dashboard shall allow admins to view, search, and manage user accounts, job listings, community posts, and reports.
- 6.3 The system shall allow admins to suspend, delete, or warn users who violate community rules.
- 6.4 The system shall allow admins to generate analytical reports on user engagement, active listings, and community interactions.
- 6.5 The system shall allow admins to enable or disable specific features (e.g., messaging, posting, or reporting) globally.
- 6.6 The system shall provide admins with tools to review user-submitted reports or support tickets and take appropriate action.

6. Notifications

- 7.1 The system shall notify users when their job application is accepted or rejected.
- 7.2 The system shall notify users when they receive a new direct message.
- 7.3 The system shall notify users when an admin responds to a report or request.
- 7.4 The system shall allow users to view their notification history from within their profile.
- 7.5 The system shall ensure all notifications are timestamped and stored in the database.

7. Security and Privacy

- 8.1** The system shall encrypt all user passwords and sensitive information using industry-standard encryption algorithms.
- 8.2** The system shall ensure all communication between client and server occurs over HTTPS.
- 8.3** The system shall implement authentication and authorization mechanisms to restrict access to specific functionalities based on user roles (admin, registered user, guest).
- 8.4** The system shall allow users to control which personal information (email, phone, profile picture, etc.) is visible to others.
- 8.5** The system shall ensure that only authorized admins can access and manage the admin dashboard.

Non-functional Requirements

This section defines the system's properties and constraints, such as its reliability, response time, and security. It is crucial that these requirements are met, because if they are not, the website could be unusable regardless of its features. We've broken these down into three main categories as per standard software engineering practice.

Product Requirements

Performance requirements:

The system shall provide a responsive user experience under normal usage conditions. Common user actions (such as posting a job, applying for a job, or sending a message) shall be processed quickly. The job feed and community posts shall update in a timely manner.

Space requirements:

The complete deployed web application shall be optimized for a reasonable deployment size. The system shall store user, job, and message data efficiently so that the database can support a large number of users and job listings. Uploaded files shall be limited to a maximum size.

Scalability requirements:

The website's architecture should be designed to handle a reasonable growth in the number of users and tasks without requiring a major redesign. The system should allow for future expansion if needed.

7.0.1 Dependability requirements

Reliability:

The system shall be designed to maintain stable operation during regular use. If an error or crash occurs, the server shall automatically restart and restore normal operation in a timely manner. The system shall minimize data loss by saving all submitted data immediately to the database.

Availability:

The website should aim to be available for users most of the time. Planned downtime for maintenance should be kept to a minimum and scheduled during off-peak hours.

Robustness:

The system shall handle invalid user input gracefully without crashing or displaying raw error messages. All user inputs shall be validated both on the client side and server side. The system shall display user-friendly error notifications when invalid actions occur and continue operating normally.

7.0.2 Security requirements

Authentication:

User passwords shall be securely stored using a modern hashing algorithm.

Authorization:

Basic input validation and authentication measures shall be implemented to protect against unauthorized access and data tampering. Role-Based Access Control will ensure users cannot access data or functions outside their permissions.

7.0.3 Usability requirements

Ease of Use:

A new user shall be able to register and complete their primary task (either posting or applying for a job) quickly and without guidance. The interface shall be intuitive and consistent, with clear navigation and prompts. The system shall include a Help/FAQ section covering key topics.

Organizational requirements

7.1.1 Development requirements

Development Methodology:

The team shall follow a simplified **iterative approach**, using short development sprints. Each sprint shall include planning, progress tracking, and review sessions to ensure steady progress and collaboration.

Version Control:

All project source code shall be maintained using a **version control system**. The repository shall include a clear branching structure to manage collaboration.

Code Documentation:

The codebase shall include clear and concise inline comments, a project README file, and basic API documentation describing the key endpoints. A short deployment setup guide shall also be included.

Testing Requirements:

Core functionalities shall be tested manually and through basic unit testing. Each new feature must be tested before deployment to ensure that it works as intended and does not affect existing components.

7.1.2 Operational requirements

User Support:

The system shall provide a simple **Help & FAQ page** covering common user questions. This will serve as the main form of user support.

Data Backup:

The system shall perform regular backups of the main database. Backups shall be stored securely and verified periodically to ensure data recoverability.

7.1.3 Environmental requirements

Portability:

The web application shall be deployable on multiple platforms. The project shall include setup instructions or optional **containerization support** for easy deployment.

External requirements

These requirements arise from factors external to the system, such as legal standards and the need to integrate with other software.

7.2.1 Interoperability requirements

Mapping API:

The system shall integrate with a **third-party map API** to display job locations and nearby opportunities within the platform.

7.2.2 Legislative requirements

Privacy Compliance:

The system shall respect user privacy by securely storing personal data. Users shall have the option to hide personal details from public view.

Accessibility Standards:

The interface shall be accessible by supporting basic accessibility features and following clear design principles.

7.2.3 Ethical requirements

Content Moderation:

Admins shall be responsible for reviewing reported posts and comments. All content moderation actions shall follow the platform's community guidelines to ensure a respectful and safe environment.

7.2.4 Regulatory requirements

Browser Compatibility:

The web application shall function correctly on the latest versions of major browsers, across both desktop and mobile devices.

Scenarios

Scenario 1: User Posts a Job Listing

Initial Assumption:

User is logged into their account. The user's profile has verified basic information (name, email, phone number, and location). User is on the "Post a Job" page

intending to create a new job listing.

Normal Flow:

1. User fills in the job form, including title, description, category (e.g., pet-sitting, tutoring), location, pay rate, and duration (one-time or part-time).
2. User can optionally upload a photo or document related to the job.
3. User reviews the preview of the listing and confirms to post.
4. System validates the inputs and adds the job post to the database.
5. The post becomes visible to other users under “Available Jobs.”
6. User can view, edit, or delete their post through their dashboard.

What Can Go Wrong:

- User input fields are incomplete or invalid (e.g., no title, invalid pay rate). The system displays an error and prevents posting until corrected.
- Network or server error prevents posting. The system saves the draft and notifies the user to retry later.
- User account is under suspension or verification pending. Posting is blocked, and user is informed.
- Duplicate or spam-like post detected — system flags it for admin review.
- User uploads an unsupported file type — system rejects file and requests correction.

Other Activities:

Admins may review or moderate the post before or after publication. The system may notify users who have alerts set for similar job types.

System State on Completion:

Job post is saved in the database and visible on the “Available Jobs” page. A record of the post creation is added to the system log (timestamp, user ID, post ID).

Scenario 2: User Applies for a Job

Initial Assumption:

User is logged in and viewing available job listings. The job post is active and visible. The job has not reached its applicant limit.

Normal Flow:

1. User clicks on the job post to view full details.
2. User selects “Apply Now.”
3. System prompts for confirmation or allows the user to add a short message or introduction.
4. System logs the application, notifies the job poster (client), and marks the job as “applied” in the user’s dashboard.
5. Job poster can review applications and choose to accept or reject.

What Can Go Wrong:

- Job post no longer available (deleted or filled). System displays a message and prevents application.
- User tries to apply more than once. System prevents duplicate submissions.
- User profile incomplete or missing essential info (e.g., contact details). System prompts user to complete profile.
- Server or connection error prevents submission — user is notified and asked to retry.
- The job poster has blocked the applicant — application is automatically rejected.

Other Activities:

Admin may monitor application activity for spam or abuse. The system may recommend similar jobs to the applicant afterward.

System State on Completion:

Application record added to the database (timestamp, job ID, applicant ID). Notifications sent to both applicant and job poster. System log updated.

Scenario 3: Admin Moderates the Platform

Initial Assumption:

Admin is logged into the admin dashboard with proper privileges. The system is running normally.

Normal Flow:

1. Admin navigates to the dashboard sections (Users, Job Posts, Community, Reports, etc.).
2. Admin reviews flagged content (e.g., job posts, comments, or reviews).
3. Admin can approve, edit, hide, or delete content as needed.
4. Admin can view user reports, generate analytics on engagement, and manage system settings (e.g., enable/disable features).
5. All actions are logged with timestamps and admin ID.

What Can Go Wrong:

- Admin privileges expire or are insufficient. Access denied; system logs the attempt.
- Database connection fails while loading content — system notifies the admin.
- Admin accidentally deletes or edits the wrong record — system provides a confirmation prompt and recovery option.
- Generated report fails due to large data volume — system suggests narrowing the range.

Other Activities:

Admins can communicate with users for clarification or warnings. The system may send automated alerts if policy violations increase.

System State on Completion:

Admin actions are recorded in system logs (who did what and when). Changes are immediately reflected on user-facing pages. Reports and metrics are stored for future reference.

Scenario 4: User Interacts on Community Page

Initial Assumption:

User is logged in and has an active profile. The community feature is enabled. User is on the “Community” page.

Normal Flow:

1. User clicks “Create Post.”
2. User writes a short message (e.g., job experience, advice, or update), and can attach an image or tag a location.
3. User submits the post. The post appears on the community feed.
4. Other users can like, comment, or report the post.
5. System tracks engagement metrics (likes, comments, shares).

What Can Go Wrong:

- Post contains prohibited or offensive content — system flags it for moderation.
- Attachment file too large or unsupported — upload rejected.
- Network error while posting — system retries or saves the draft locally.
- User’s account is restricted — posting is blocked.
- Commenting disabled temporarily by admin — user is notified.

Other Activities:

Admins may monitor or moderate discussions. Users may follow or message others directly from their posts.

System State on Completion:

Post and engagement data are saved in the database. Activity is logged (user ID, timestamp, post ID). Notifications are sent to followers or mentioned users.

Scenario 5: User Reviews Applicants and Selects Someone for Their Posted Job

Initial Assumption:

User (job poster) is logged in and has one or more active job listings. Applicants have already applied to at least one of their posts.

Normal Flow:

1. User navigates to their dashboard → “My Job Posts.”
2. User selects a job to view all applicants who have applied.
3. System displays a list of applicants with their profile previews (name, rating, short bio, and contact info if public).
4. User clicks an applicant to view their full profile and application message.
5. User decides to:
 - **Accept the applicant:** System updates the job status to “In Progress” and notifies both parties.
 - **Reject the applicant:** System sends a polite rejection notification.
 - **Contact the applicant:** User can message them directly via in-app chat, email, or phone (depending on privacy settings).
6. Once the job is filled, the user can mark it as “Closed.”

What Can Go Wrong:

- Applicant profile unavailable (deleted or suspended) — system removes it from the applicant list.
- Job listing accidentally closed — system prevents new selections until reopened.
- Messaging service temporarily unavailable — system suggests alternate contact (email).
- Multiple users accepted simultaneously due to lag — system only confirms the first successful action.
- User violates communication policies (e.g., sharing prohibited info) — system flags and warns the user.

Other Activities:

Admin may oversee applicant selection and review flagged communications. The system may recommend similar applicants or autofill past message templates.

System State on Completion:

Job status is updated to “In Progress.” The selected applicant and job poster are linked in the system records. Notifications are sent to both parties, and a log entry is created with timestamp and user IDs.

Scenario 6: User Gets Accepted for a Job and Begins Communication

Initial Assumption:

User is logged in and has applied for one or more job listings. One of their applications has been accepted by a job poster.

Normal Flow:

1. System sends an in-app and/or email notification: “Your application has been accepted!”
2. User views the notification and is redirected to the job details page.
3. Status changes to “Accepted.”
4. User can now view the employer’s contact info (based on privacy settings) and access a chat option for direct communication.
5. User sends an introductory message confirming interest and schedules details (e.g., time, location, payment method).
6. After completing the job, the user can mark it as “Completed” and leave a review for the employer.

What Can Go Wrong:

- Notification not delivered due to email/spam filter — user may check the dashboard manually.
- Employer revokes acceptance before contact — system updates the status and notifies the user.

- Messaging feature temporarily unavailable — user can fall back to alternate contact info.
- Miscommunication (wrong time/date) — system keeps message logs for verification.
- Employer account suspended mid-process — job auto-canceled, and user notified.

Other Activities:

Admin may monitor or moderate messages. The system may prompt both sides for reviews after job completion.

System State on Completion:

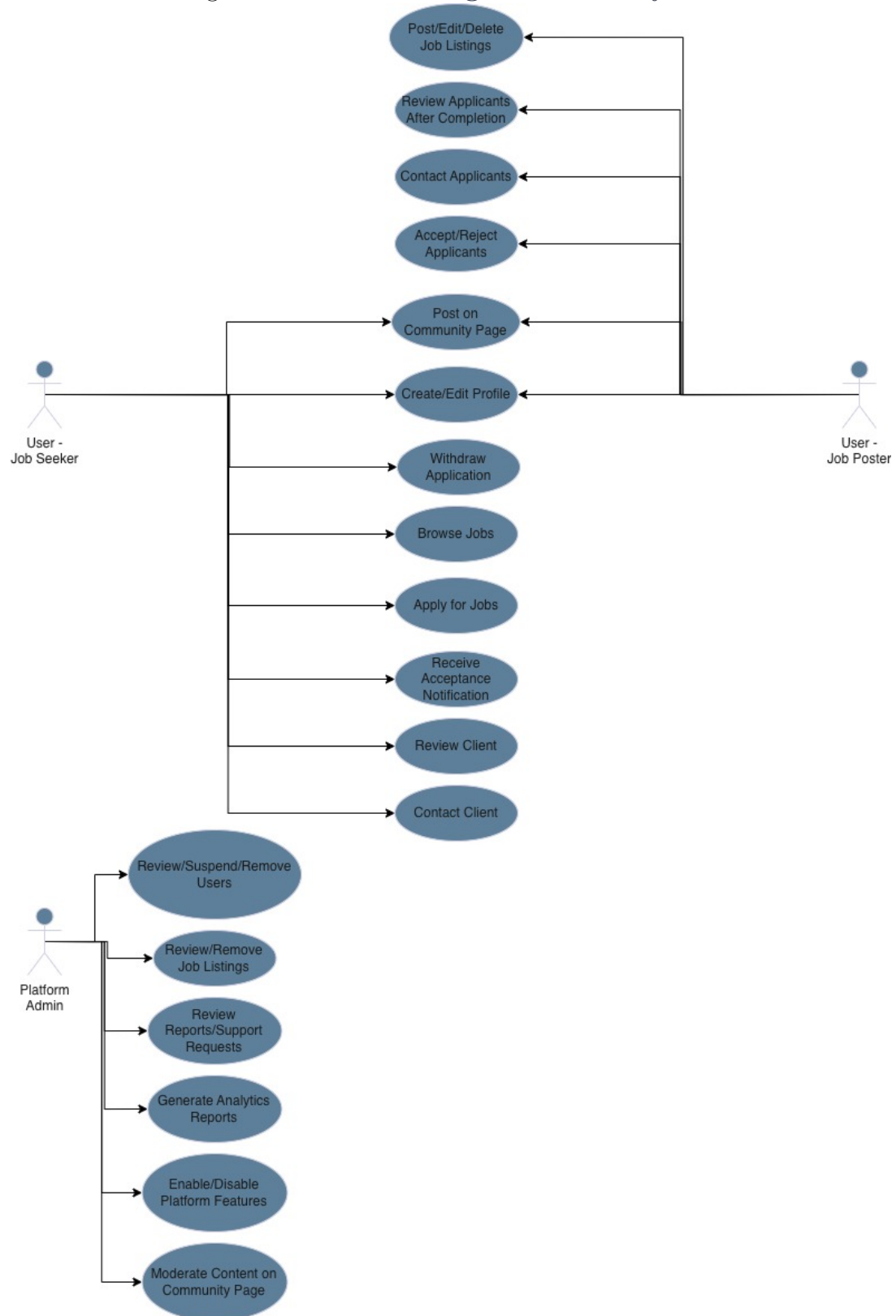
Application record is updated to “Accepted.” Both users gain temporary communication access. Activity is logged (timestamp, job ID, user IDs). If the job is completed later, a completion record and mutual reviews are added to the database.

Use Cases

Actors:

- **User:** Job Poster / Applicant
- **Admin:** Platform moderator

Figure 1: Use Case Diagram for the System



Form-Based Specification

Sign Up / Sign In

Aspect	Details
Definition	Enables users to either create a new account (sign up) or access an existing one (login).
Inputs	<i>Sign Up</i> : Name, email, age, password. <i>Login</i> : Email, password.
Outputs	<i>Sign Up</i> : Account created confirmation message, new record in user database. <i>Login</i> : Successful authentication message, redirect to home/profile page.
Requires	Connection to user database; validation for email format, password strength, and credential matching.
Action	<i>Sign Up</i> : Validate inputs → create and store new user record → send confirmation email → redirect to login/home page. <i>Login</i> : Validate credentials → verify against stored records → grant access if valid and redirect to home/profile page or display error message.
Precondition	<i>Sign Up</i> : User does not already have an account. <i>Login</i> : User must have an existing registered account.
Postcondition	<i>Sign Up</i> : New user record successfully created in the system. <i>Login</i> : User is authenticated and logged into their account.
Side Effects	<i>Sign Up</i> : Confirmation email sent to the user. <i>Login</i> : -

Post a Job

Aspect	Details
Definition	Allows a user to create and publish a new job listing on the platform.
Inputs	Job title, description, category, location, payment amount, estimated duration.
Outputs	A new job post is created and becomes visible in public job listings, on the map, and in the “My Posted Jobs” section of the user’s profile. A confirmation message is shown.
Requires	Access to database and mapping service.
Action	User fills form → system validates data → new job record is created and stored → post appears publicly.
Precondition	User is logged in with a valid account.
Postcondition	A new active job listing exists in the system and is linked to the user’s profile.
Side Effects	Platform analytics for job categories are updated.

View My Posted Jobs

Aspect	Details
Definition	Allows a user to view all job listings they have created and posted on the platform.
Inputs	User Profile
Outputs	A list showing job title, description, payment, posting date, job status, number of applicants, and management options.
Requires	Access to database.
Action	User navigates to “My Posted Jobs” → system fetches all jobs where user is the poster → displays them in a detailed list with management options.
Precondition	User is logged in and has posted at least one job.
Postcondition	User can see comprehensive details and manage all their created job listings.
Side Effects	-

Accept / Decline Job Requests

Aspect	Details
Definition	Allows a job poster to review and accept or decline applications received for their posted job.
Inputs	User selection (Accept / Decline) for a specific applicant.
Outputs	Selected applicant receives acceptance/declination notification; job status updates accordingly.
Requires	Access to database and notification system.
Action	Job poster reviews applicants → selects Accept / Decline → system notifies applicant and updates job status.
Precondition	User must be logged in and have a posted job with pending applications.
Postcondition	Application status is updated; accepted applicant is assigned to the job.
Side Effects	Accepted job cascades to applicant's "Jobs I Applied For" section.

Search / Browse Jobs

Aspect	Details
Definition	Allows users to find and discover job opportunities using filters and search terms.
Inputs	Search keywords, category filters, location, price range, job type.
Outputs	List of matching jobs with basic details (title, price, location, poster rating).
Requires	Database access and query functionality.
Action	User enters search / filters → system queries database → returns matching jobs in list / map view.
Precondition	User must be logged in.
Postcondition	User can view and apply to relevant job opportunities.
Side Effects	Personalized job recommendations improve based on user activity.

Direct Messages

Aspect	Details
Definition	Allows users to send and receive private text messages with other users on the platform.
Inputs	Message content (text), recipient selection.
Outputs	Message delivered to recipient's inbox and displayed in the chat conversation.
Requires	Database and notification system access.
Action	User composes message → selects recipient → sends → system stores and delivers message.
Precondition	Both users must have active accounts; sender must be logged in.
Postcondition	New message thread or message created between users.
Side Effects	Unread message count increases for recipient; chat history updates.

Posting a Community Post

Aspect	Details
Definition	Allows users to create and share posts on the community feed to share experiences or updates.
Inputs	Post content (text, optional image).
Outputs	Post appears in community feed; confirmation message shown.
Requires	Database and media storage access.
Action	User writes content → optionally adds image → posts → system publishes to feed.
Precondition	User must be logged in with active account.
Postcondition	New post visible in the community feed.
Side Effects	Post appears in followers' feeds; engagement metrics updated.

Report an Issue

Aspect	Details
Definition	Allows users to report problematic content, technical issues, or other users for admin review.
Inputs	Issue type, related job (if applicable), person involved (if applicable), description, evidence (optional).
Outputs	Confirmation message; report submitted to admin dashboard.
Requires	Database and admin notification system access.
Action	User selects issue type → provides details → submits → system creates report record for admins.
Precondition	User must be logged in with active account.
Postcondition	Report created and queued for admin review.
Side Effects	Reported user's rating decreases temporarily pending review.

Remove Inappropriate Content

Aspect	Details
Definition	Allows admins to permanently remove content that violates platform guidelines.
Inputs	Content to remove, type of violation.
Outputs	Content removed; removal confirmation shown to admin.
Requires	Admin privileges; access to content database.
Action	Admin reviews reported content → selects removal → chooses violation type → system deletes content.
Precondition	Admin logged in with appropriate permissions; content must violate guidelines.
Postcondition	Content permanently removed.
Side Effects	Content statistics updated; repeat offenses may trigger suspension.

Tabular Specification

Tabular Specification of Main System Functions

Condition (Input / State)	System Action / Response	Result / Outcome
User submits valid sign-up information (email, age, password).	System validates inputs, stores user data, and sends a confirmation email.	User redirected to login or home page.
User enters correct login credentials.	System verifies credentials and initializes a user session.	User is logged in and directed to their home profile.
User posts a new job with all required details.	System validates job fields, stores them in the database, and links them to the user's profile.	Job becomes visible publicly and in "My Posted Jobs."
User views "My Posted Jobs."	System retrieves all jobs posted by that user from the database.	User can view, edit, or close any active post.
Job poster accepts or declines an applicant.	System updates the job's application status and sends notifications to applicants.	Accepted applicants are assigned; others receive decline messages.
User sends a direct message to another user.	System stores and delivers the message to the recipient's inbox in real time.	Message appears in both users' chat threads; unread count updates.
User writes and submits a new community post (text and/or image).	System validates the post content, uploads the image if included, and saves the post to the community database.	Post is successfully published to the community feed and visible to followers.
User submits a report including issue type, description, and optional evidence.	System validates inputs, records the report in the database, and alerts the admin dashboard for review.	Report successfully submitted; user receives confirmation message.

Report involves another user's content or behavior.	Admin reviews the report, contacts both users to clarify and resolve the issue, and if confirmed, manually reduces the reported user's rating.	Report resolved; users notified and ratings updated.
Admin identifies and confirms content violating guidelines.	System removes the content, notifies the creator, and logs the action for record-keeping.	Inappropriate content removed; creator informed; system updated.